A CASE-STUDY BASED REPORT

ON

LINUX COMMAND EXECUTION TIME PROFILER

submitted in partial fulfilment of the requirements for the Course

OPERATING SYSTEMS

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Team Shell Forge

Name Dharmika P	**roll 2520030012**
Name Rushvini	**roll 2520030542**
Name Anushka	**roll 2520030501**

Under the Esteemed guidance of

Mrs. Harika

Asst.Professor, Dept. of CSE

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Linux Command Execution Time Profiler

Measure the execution time and resource usage of any Linux command

Table of Contents

1. Problem Statement
2. System Design
3. Implementation Approach
4. Technologies Used
5. Testing
6. Results
7. Impact and Future Scope

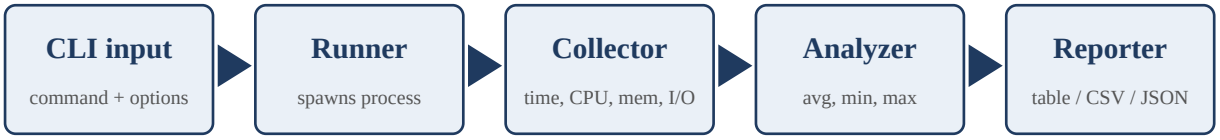
1. Problem Statement

The aim of this project is to create a tool that measures the execution time and resource usage of Linux commands. Existing options fall short in the following ways:

Limited visibility	The built-in *time* command shows only basic numbers and is hard to compare across runs.
Scattered tools	Time, CPU, memory and I/O metrics live in different tools, so profiling is slow.
Hard to compare	Single runs are noisy; developers need repeated runs and clear summaries.

2. System Design

The profiler follows a five-stage pipeline, processed from left to right:



Input	User runs: `profiler [options] "command"`. Options set the repeat count and output format.
Output	Per-run and averaged metrics, shown in the terminal or saved to a file.

3. Implementation Approach

The tool is implemented in four steps:

1. Parse input	Read the command, repeat count (`-n`) and format (`-f`) from the command line.
2. Run and time	Launch the command as a child process; record wall-clock start and end time.
3. Collect usage	Read user/system CPU time, peak memory (max RSS), I/O and context switches from the kernel.
4. Report	Repeat N times, compute average, min and max, then print a table or export CSV/JSON.

4. Technologies Used

Python 3	Core logic, argument parsing (argparse) and report formatting.
subprocess + resource	Run commands and read CPU time and peak memory of child processes.
psutil / /proc	Live CPU, memory and I/O statistics from the Linux kernel.
Linux + Git	Developed and tested on Ubuntu; versioned with Git and GitHub.

5. Testing

Each type of test validates a different aspect of the tool:

Functional tests	Quick commands (*ls*, *echo*), failing commands and invalid input.
Workload tests	CPU-heavy (*sort*, *gzip*), I/O-heavy (*cp*, *dd*) and sleep-based commands.
Accuracy check	Compared results with `/usr/bin/time -v` to confirm they match.
Repeatability	Multiple runs per command to confirm stable averages.

6. Results

Command	Wall time	CPU time	Peak memory
ls -l /usr	0.01 s	0.01 s	3 MB
sort bigfile.txt	1.80 s	1.65 s	120 MB
gzip bigfile.txt	2.40 s	2.30 s	8 MB
sleep 2	2.00 s	0.00 s	2 MB

Note: sample output for illustration; replace with your own measured values.

Key observations:

- Matches `/usr/bin/time` results for all test commands.
- Clearly separates CPU-bound, I/O-bound and idle commands.
- Averages over repeated runs give stable, comparable numbers.

7. Impact and Future Scope

Faster debugging	Quickly spot slow or memory-hungry commands and scripts.
Better decisions	Compare alternative commands with real data instead of guesses.
Learning value	Helps students understand processes, CPU, memory and I/O in Linux.
Future scope	Live graphs, a web dashboard, and per-process history tracking.

Thank you.